



RAPPORT PROJET : MLOPS — GRAVITÉ DES ACCIDENTS DE LA ROUTE

Mentor : Kilyan

Jacques CATTELIN – Noël CHING

MAR25_CMLE_ACCIDENT

Octobre 2026



SOMMAIRE

Introduction	4
1. Définition du produit	5
1.1 Vision produit	5
1.2 Les personas	6
1.3 Synthèse du cadrage produit.....	6
2. Données et modèle.....	8
2.1 Les données ONISR.....	8
2.1.1 Source et périmètre	8
2.1.2 La cible : un accident grave.....	8
2.1.3 Les 27 variables du modèle	8
2.2 La chaîne ETL.....	8
2.2.1 Principe : corriger plutôt qu'arrêter	8
2.2.2 Les quatre étapes	9
2.3 Modélisation et entraînement.....	9
2.3.1 Découpage temporel	9
2.3.2 Trois algorithmes comparés à chaque cycle.....	9
2.3.3 Le choix du champion	9
2.3.4 Résultats des quatre cycles.....	10
2.4 Développement et blueprint.....	10
3. La solution MLOps	11
3.1 Flux MLOps & collaboratifs	11
3.2 Architecture de la solution	12
3.2.1 La stack technique	13
3.2.2 Où sont le code et les données	14
3.3 Intégration, déploiement et tests	14
3.4 Résilience.....	15
3.5 Traçabilité : Git × DVC × MLflow	15
3.6 Monitoring et alertes.....	15
3.6.1 Architecture de supervision	15
3.6.2 Les 7 dashboards et les 12 alertes.....	16
3.6.3 La dérive des données (Evidently)	16
3.7 Sécurité.....	17
3.8 Les applications.....	17
3.8.1 Le Cockpit d'administration.....	17
3.8.2 Le Cockpit public.....	18
3.8.3 La documentation	19
3.9 Démonstrations	19
4. Pour aller plus loin	20
Conclusion	21
Bibliographie	22
Annexes	23
A. La documentation du Cockpit.....	23
B. Chiffres clés	23
C. Glossaire	24

PRÉAMBULE

Ce rapport présente le travail réalisé dans le cadre du projet **CAC MLOps** : la mise en production, l'exploitation et la surveillance d'un modèle de Machine Learning qui estime la **gravité des accidents de la route** à partir des données publiques de l'ONISR.

La première partie, **Définition du produit**, présente la vision produit, les personas et leurs besoins. Elle formalise l'objectif auquel la solution doit répondre : une application toujours disponible pour ses utilisateurs.

La deuxième partie, **Données et modèle**, décrit les données ONISR, la chaîne ETL qui les charge chaque année, les 27 variables du modèle et la manière dont trois algorithmes sont entraînés, comparés et promus.

La troisième partie, **La solution MLOps**, est le cœur du projet : les flux de mise en production, l'architecture, les tests, les mécanismes de résilience, la traçabilité, le monitoring, la sécurité et les applications livrées.

Le contenu de ce rapport reprend volontairement la **documentation du Cockpit** (18 documents, onglet Docs), maintenue au fil du projet : les chiffres et les règles cités ici sont ceux du code et de la production. Les trois schémas de synthèse, cœur de la présentation de soutenance, sont repris avec leur explication orale.

Comme pour tout projet de ce type, le travail a été mené de manière itérative : chaque incident rencontré en production (coupure, déploiement en échec, donnée inattendue) a donné lieu à un correctif et à une règle documentée.

Introduction

Chaque année, l'Observatoire national interministériel de la sécurité routière (ONISR) publie sur data.gouv.fr la base des accidents corporels de l'année écoulée : environ **55 000 accidents**, décrits par quatre fichiers (caractéristiques, lieux, usagers, véhicules). Ces données permettent d'entraîner un modèle qui estime si un accident fera au moins une victime **grave** (blessé hospitalisé ou tué).

Entraîner un tel modèle sur un poste de travail ne pose pas de difficulté particulière. Le faire vivre en production est un autre problème : intégrer chaque nouvelle année sans casse, comparer honnêtement les nouveaux modèles à celui en place, déployer sans interrompre le service, revenir en arrière en cas d'échec, détecter une dérive des données, et garder la trace de chaque version. C'est précisément l'objet du MLOps.

Ce projet construit cette chaîne de bout en bout, sur une infrastructure réelle : un serveur VPS Scaleway qui héberge la production, un cluster Kubernetes (Kapsule) qui assure la haute disponibilité de l'accès public, un stockage objet S3, et GitHub pour le code et l'intégration continue. Trois événements déclenchent une mise en production — une nouvelle année de données, un nouveau code, un nouveau modèle proposé par un data scientist — et tous passent par le même chemin contrôlé, avec une **validation humaine** avant toute interruption de service.

Le modèle en production, *lgbm_accidents v4*, est un LightGBM entraîné sur 2021-2023 et évalué sur 2024 (F1 = 0,689, AUC = 0,837, rappel = 0,767). Il est servi par une API et par deux Cockpits Gradio, et surveillé en continu par Prometheus, Loki et Grafana.

1. Définition du produit

1.1 Vision produit

Le produit est une **chaîne MLOps** qui met à disposition un modèle de prédiction de gravité, et le maintient à jour, fiable et sécurisé dans le temps. Elle opère deux environnements de production complémentaires : un **VPS** (accès API, Cockpit public et Cockpit d'administration) et un cluster **Kubernetes** (accès API et Cockpit public), allumé à la demande pour la haute disponibilité.

La vision produit tient en une phrase : **avoir une application toujours fiable et disponible pour Julie**, l'utilisatrice finale. Pour l'atteindre, la chaîne repose sur trois principes :

- **Fiabilité** : chaque changement est testé avant et après sa mise en production, et la version précédente est restaurée automatiquement en cas d'échec.
- **Automatisation** : l'ingestion des données, l'entraînement, la comparaison des modèles, le déploiement et les tests s'enchaînent sans intervention, sauf la décision de mise en production, qui reste humaine.
- **Sécurité** : les outils d'administration ne sont joignables que par un VPN, l'accès public est chiffré et protégé contre les abus, les images sont scannées.

Schéma de synthèse n°1 — Vision produit & personas

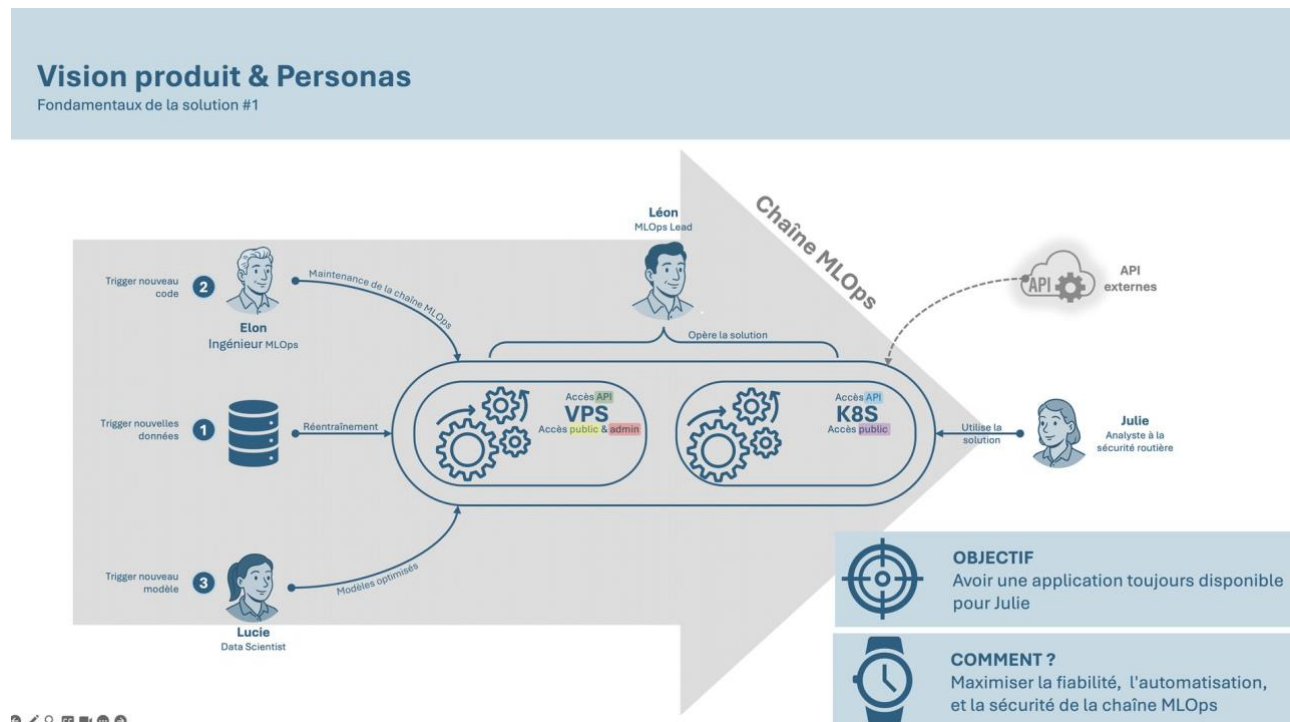


Figure 1 – Vision produit & personas (Fondamentaux de la solution #1)

Ce que dit ce schéma

« Ce premier schéma pose le décor. Au centre, la grande flèche, c'est la chaîne MLOps : elle opère deux environnements de production. À gauche le VPS, qui expose l'API, le Cockpit public et le Cockpit d'administration ; à droite Kubernetes, qui expose l'API et le Cockpit public, et qui apporte la haute disponibilité. »

« À droite, ceux qui utilisent la solution : Julie, analyste à la sécurité routière, qui travaille dans le Cockpit public, et des applications externes qui appellent l'API. À gauche, les trois événements qui font évoluer la solution : (1) de nouvelles données publiées par l'ONISR, qui déclenchent un réentraînement ; (2) Elon, l'ingénieur MLOps, qui maintient la chaîne ; (3) Lucie, la data scientist, qui propose des modèles optimisés. Au-dessus, Léon, le MLOps Lead, opère la solution : c'est lui qui valide chaque mise en production. »

« L'objectif est simple : une application toujours disponible pour Julie. Le comment tient en trois mots : fiabilité, automatisation, sécurité. La suite de la présentation montre comment chacun de ces trois mots est tenu. »

1.2 Les personas

La solution sert quatre personas, qui correspondent chacun à un rôle dans la chaîne et à un point d'entrée dans les outils :

Persona	Rôle	Ce qu'il fait dans la solution	Outil principal
Julie	Analyste à la sécurité routière — utilisatrice finale	Estime la gravité d'un accident, simule l'effet d'une mesure (giratoires, zone 30...), repère les zones à risque	Cockpit public (VPS ou Kubernetes)
Léon	MLOps Lead — opère la solution	Valide ou refuse chaque mise en production (GO / STOP), surveille la santé, la dérive et les modèles, déclenche les flows	Cockpit admin, Grafana, Prefect
Elon	Ingénieur MLOps — maintient la chaîne	Fait évoluer le code de la chaîne sur la branche mlops (Trigger 2)	GitHub, CI/CD
Lucie	Data scientist — optimise le modèle	Explore de nouveaux hyperparamètres sur la branche DS et propose un blueprint (Trigger 3)	MLflow (expérience dev), GitHub

Tableau 1 — Les personas de la solution

Zoom sur l'utilisatrice finale : Julie

Julie analyse l'accidentalité pour orienter les politiques de sécurité routière. Elle n'est pas data scientist : elle attend des résultats lisibles, sur des données à jour, et un outil qui répond quand elle en a besoin.

- **Objectifs** : estimer le risque d'un type d'accident, comparer l'effet de mesures avant de les proposer, cibler les zones où agir en priorité.
- **Difficultés** : des données brutes volumineuses et codées (nomenclatures ONISR), aucune visibilité sur la fiabilité d'un modèle.
- **Besoins** : une application simple et toujours disponible, des simulations expliquées, des données de l'année la plus récente.

1.3 Synthèse du cadrage produit

Le produit doit transformer une publication annuelle de données brutes en un service de prédiction fiable et explicable, et le faire vivre dans le temps sans dégrader l'expérience de l'utilisatrice. Les exigences qui en découlent structurent toute la solution :

Exigence	Traduction dans la solution
Toujours disponible	Deux environnements de production ; Kubernetes sans interruption lors des déploiements ; disponibilité mesurée de l'extérieur toutes les 15 s
Données à jour	Détection automatique chaque lundi de la publication ONISR, chargement et réentraînement automatiques
Modèle fiable	3 algorithmes comparés à chaque cycle, seuils minimums, comparaison au modèle en place, validation humaine
Changements maîtrisés	Un seul chemin vers la production pour les 3 déclencheurs, tests avant et après, rollback automatique
Traçabilité	Chaque modèle relié à son code (Git), à ses données (DVC) et à son entraînement (MLflow)
Sécurité	Administration par VPN uniquement, HTTPS, limitation anti-abus, scan des images, secrets rotatifs

Tableau 2 — Exigences produit et réponses de la solution

La partie suivante décrit les données et le modèle ; la troisième partie montre comment la chaîne MLOps tient ces exigences.

2. Données et modèle

2.1 Les données ONISR

2.1.1 Source et périmètre

Les données proviennent des **bases annuelles des accidents corporels de la circulation routière** publiées par l'ONISR sur data.gouv.fr. Chaque année comprend quatre fichiers CSV liés par l'identifiant de l'accident (*Num_Acc*) : *caracteristiques* (date, lieu, conditions), *lieux* (route), *usagers* (victimes) et *vehicules*. Le périmètre actuel couvre **2021 à 2024**.

Année	Accidents	Usagers	Véhicules	Taille	Versionnée par
2021	56 519	129 154	97 316	31,1 Mo	Manuel (mise en place)
2022	55 303	126 663	94 494	31,5 Mo	Manuel (mise en place)
2023	54 823	125 790	93 586	34,1 Mo	Manuel (mise en place)
2024	54 403	125 188	92 678	33,8 Mo	Automatique (ETL Prefect)

Tableau 3 — Volumétrie des données brutes ONISR (source : doc « Lignée des données »)

L'année 2024 est la première intégrée de bout en bout par la chaîne, sans intervention humaine. L'année 2025 sera détectée automatiquement à sa publication (attendue vers octobre 2026) : elle deviendra l'année de test et 2024 rejoindra l'ensemble d'entraînement.

2.1.2 La cible : un accident grave

La gravité est évaluée pour la **victime la plus touchée** de chaque accident. La cible binaire *grav* vaut **1** si au moins un usager est hospitalisé ou tué, **0** s'il est indemne ou blessé léger. Un accident impliquant plusieurs usagers est ramené à une ligne : celle de la victime la plus grave. Les accidents dont aucun usager n'a de gravité renseignée (60 en 2021, 241 en 2022, 118 en 2023) sont écartés et journalisés.

2.1.3 Les 27 variables du modèle

Catégorie	Variables	Exemples de modalités
Temporelles (3)	mois, jour, hour	jour 1 = lundi ; heure 0–23 extraite de hrnn
Géographiques (5)	dep, com, lat, long, agg_	Corse 2A/2B → 201/202 ; agg_ 1 = hors agglomération
Conditions (4)	lum, atm, col, int	lum 3 = nuit sans éclairage ; int 6 = giratoire ; atm recodé en binaire
Voirie (5)	catr, circ, surf, situ, vma	catr 1 = autoroute ; surf 7 = verglacée ; vma 10–130 km/h
Usager (5)	place, catu, sexe, secu1, victim_age	catu 3 = piéton ; âge = année – année de naissance
Véhicule (3)	catv, obsm, motor	catv : 40 modalités regroupées en 7 classes
Agrégées (2)	nb_victim, nb_vehicules	nombre de victimes et de véhicules de l'accident

Tableau 4 — Les 27 variables du modèle (source : doc « Dictionnaire des données »)

L'année de l'accident (*year_acc*) sert de clé au découpage temporel mais n'est pas une variable du modèle, pour éviter toute fuite d'information temporelle.

2.2 La chaîne ETL

2.2.1 Principe : corriger plutôt qu'arrêter

Une source publique change : un fichier renommé, une colonne renommée, un séparateur différent, des doublons, un code hors nomenclature. La chaîne ETL est conçue pour **auto-corriger un maximum d'anomalies**

plutôt que d'alerter ou de s'arrêter, chaque correction étant tracée. Elle ne s'arrête que si la donnée est réellement inexploitable.

Comportement	Nombre	Effet
AUTO_CORRECTED	17	Anomalie connue corrigée automatiquement — le pipeline continue
AUTO_RETRY	2	Réessai automatique (réseau, DVC)
WARNING	7	Alerte email non bloquante — le pipeline continue
CRITICAL	5	Arrêt — donnée réellement inexploitable, le modèle en place reste en production

Tableau 5 — Comportements de la chaîne ETL face aux anomalies (source : doc « Catalogue ETL »)

2.2.2 Les quatre étapes

1. **Découverte et téléchargement** — l'API data.gouv.fr est interrogée à chaque exécution : aucune URL ni nom de fichier n'est codé en dur. Les fichiers sont reconnus par mot-clé puis renommés de façon canonique, ce qui élimine par construction une faute de frappe de la source (cas réel : « carcteristiques » en 2021 et 2022).
2. **Validation Pandera sur 3 niveaux** — format (fichier présent, lisible), schéma (colonnes, types : renommages connus, doublons supprimés, valeurs hors nomenclature ramenées à « non renseigné ») et qualité (volumétrie, taux de valeurs manquantes, coordonnées).
3. **Versionnage DVC** — données brutes puis données préparées poussées vers le stockage S3 Scaleway, et pointeur .dvc commité automatiquement sur *main*.
4. **Préparation** — fusion des 4 tables, recodages, imputation des valeurs manquantes par le mode calculé sur l'entraînement uniquement, puis découpage temporel : la dernière année sert de jeu de test.

Après chaque chargement, Evidently produit en complément un rapport de **qualité** des fichiers bruts (doublons, valeurs manquantes par table), affiché dans le Cockpit et dans Grafana.

2.3 Modélisation et entraînement

2.3.1 Découpage temporel

Le modèle apprend sur les **années passées** et il est évalué sur la **plus récente**, qu'il n'a jamais vue — exactement la situation de production, où il prédit des accidents futurs. Un découpage aléatoire mélangerait des accidents de la même année entre apprentissage et test et surestimerait la performance réelle. Pour le cycle 2024 : 172 693 lignes d'apprentissage (2021-2023) et 64 934 lignes de test (2024).

2.3.2 Trois algorithmes comparés à chaque cycle

Random Forest (*rf_accidents*), XGBoost (*xgb_accidents*) et LightGBM (*lgbm_accidents*) sont entraînés sur exactement les mêmes données à chaque cycle : le choix du meilleur est refait à chaque fois, il n'est jamais figé. Quatre métriques sont calculées sur l'année de test, avec des seuils minimums fixés environ 8 % sous les performances observées :

Métrique	Ce qu'elle mesure	Seuil minimum
F1 (principale)	Équilibre entre précision et rappel sur la classe « grave »	≥ 0,60
AUC ROC	Capacité à classer un accident grave au-dessus d'un non grave	≥ 0,77
Rappel	Part des accidents réellement graves détectés	≥ 0,58
Accuracy	Part de prédictions correctes, toutes classes confondues	≥ 0,72

Tableau 6 — Métriques et seuils minimums de promotion

2.3.3 Le choix du champion

Trois filtres successifs désignent le **champion** d'un cycle, puis un humain décide :

1. passer les **4 seuils minimums** (un seul seuil manqué élimine le modèle) ;
2. être le **meilleur en F1** parmi les modèles qualifiés ;
3. **faire mieux que le modèle en production**, selon une règle qui dépend du déclencheur ;
4. **validation humaine** : GO ou STOP dans le Cockpit, au vu des métriques comparées.

Déclencheur	Règle de comparaison	Pourquoi
Trigger 1 — nouvelle année	Promu si au plus 1 métrique sur 4 régresse	Le nouveau modèle est testé sur une année plus récente : la comparaison stricte n'est pas équitable
Trigger 3 — nouveau blueprint	$F1 \geq F1 \text{ de la production} + 0,01$	Mêmes données, même test : le +0,01 évite un changement dû au hasard
Aucun modèle en production	Promotion directe	Rien à comparer

Tableau 7 — Règles de promotion selon le déclencheur (source : doc « Entraînement des modèles »)

2.3.4 Résultats des quatre cycles

Cycle	Données (train → test)	Champion	F1	AUC	Rappel
1	2021 (split 70/30)	rf v1	0,701	0,843	0,765
2	2021 → 2022	lgbm v2	0,703	0,842	0,757
3	2021-2022 → 2023	lgbm v3	0,692	0,840	0,756
4 — @Production	2021-2023 → 2024	lgbm v4	0,689	0,837	0,767

Tableau 8 — Champion de chaque cycle annuel (source : doc « Lignée des données »)

Les métriques ne se comparent pas strictement d'un cycle à l'autre : le jeu de test change à chaque fois. LightGBM s'impose dès le cycle 2 ; XGBoost reste nettement en retrait sur le rappel ($\approx 0,60$), ce qui l'écarte malgré une AUC comparable.

2.4 Développement et blueprint

La data scientist explore en local, sans jamais toucher la production. Ses expériences remontent dans le MLflow partagé, dans une expérience séparée (`accidents_severity_dev`), ce qui rend le **développement tracé et partagé au même titre que la production**. Quand elle trouve de meilleurs hyperparamètres, elle les extrait dans un **blueprint** (`config/model_params.yml`), le fichier de référence que l'entraînement de production utilise à chaque cycle, et le propose par une PR relue.

	Production	Développement
Où	Prefect, sur le VPS	Poste de la data scientist
Expérience MLflow	<code>accidents_severity_prod</code>	<code>accidents_severity_dev</code>
Modèles	3 algorithmes, blueprint en vigueur, enregistrés dans le registre	Un algorithme, hyperparamètres libres, non enregistrés
Aboutissement	Champion → gate → @Production	Meilleur run → blueprint → PR (Trigger 3)

Tableau 9 — Entraînement de production et de développement

3. La solution MLOps

3.1 Flux MLOps & collaboratifs

Trois événements peuvent faire évoluer la production. Ils empruntent des chemins différents au départ, mais **convergent tous vers la même gate humaine**, puis vers le même déploiement : d'abord le VPS, puis Kubernetes si le VPS est sain, chacun suivi de tests fonctionnels.

Schéma de synthèse n°2 — Flux MLOps & collaboratifs

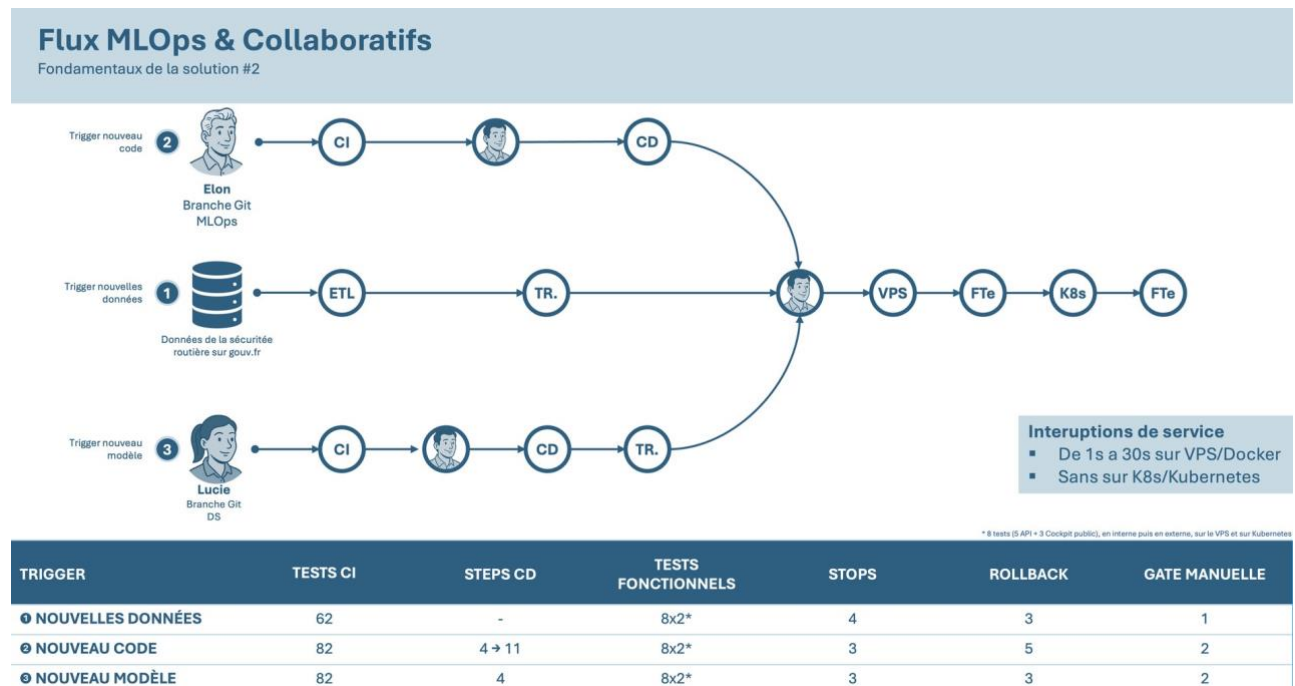


Figure 2 – Flux MLOps & collaboratifs (Fondamentaux de la solution #2)

Ce que dis ce schéma

« Ce schéma se lit de gauche à droite, une ligne par déclencheur. Ligne du milieu, le Trigger 1 : chaque lundi, la chaîne regarde si l'ONISR a publié une nouvelle année. Si oui, l'ETL charge et valide les données, puis le training entraîne les trois algorithmes et désigne un champion. »

« En haut, le Trigger 2 : Elon pousse du code sur la branche mlops. La CI le teste, je relis et je merge la PR — c'est la première validation humaine — puis le CD construit les images et les scanne. En bas, le Trigger 3 : Lucie propose un nouveau blueprint ; même CI, même relecture, puis un réentraînement avec ses hyperparamètres. »

« Les trois lignes convergent vers un seul point : la gate. Rien n'est mis en production sans mon GO. Ensuite, le déploiement se fait d'abord sur le VPS, suivi des tests fonctionnels — FTe —, puis seulement si tout est vert, sur Kubernetes, avec à nouveau des tests fonctionnels. Au moindre échec, la version précédente est restaurée automatiquement. »

« Le tableau du bas chiffre ces garde-fous : 82 tests unitaires en CI, jusqu'à 11 étapes de CD, 8 tests fonctionnels joués en interne puis en externe, des points d'arrêt, des rollbacks automatiques et une ou deux validations humaines. Côté utilisateur, une mise en production coûte au plus quelques secondes à 30 secondes d'interruption sur le VPS, et aucune sur Kubernetes. »

Trigger	Tests CI	Steps CD	Tests fonctionnels	Stops	Rollback	Gate manuelle
① Nouvelles données	62	—	8 × 2	4	3	1
② Nouveau code	82	4 → 11	8 × 2	3	5	2
③ Nouveau modèle	82	4	8 × 2	3	3	2

Tableau 10 — Garde-fous par déclencheur (8 tests = 5 API + 3 Cockpit public, en interne puis en externe, sur le VPS et sur Kubernetes)

Pour le Trigger 1, les « tests CI » correspondent aux 62 contrôles de la chaîne de données, testés en CI et exécutés pour de vrai sur les données ONISR à chaque chargement. Les validations humaines comptent la relecture-merge de la PR (Triggers 2 et 3) et le GO.

3.2 Architecture de la solution

La solution est répartie sur quatre ensembles : le **poste local** des développeurs, **GitHub** (code, CI/CD, registre d'images GHCR), et chez Scaleway le **VPS**, le cluster **Kubernetes** et le stockage **S3**.

Schéma de synthèse n°3 — Architecture de la solution

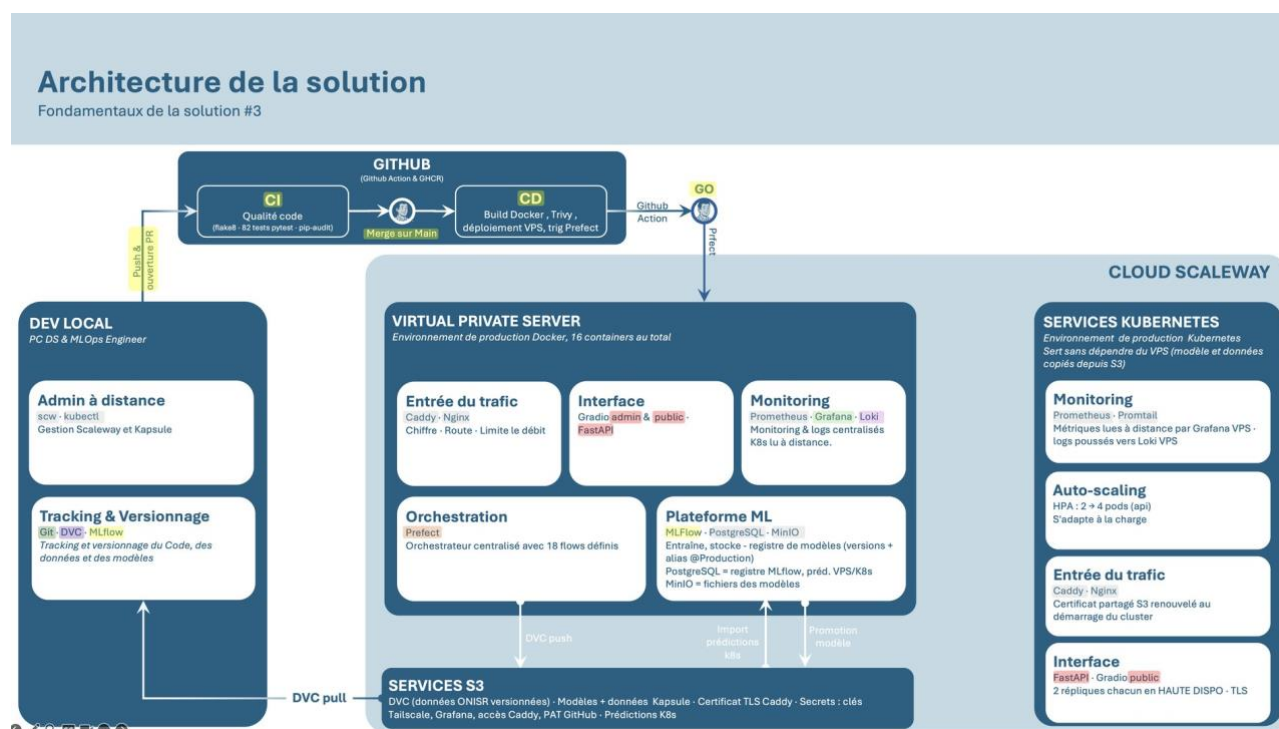


Figure 3 – Architecture de la solution (Fondamentaux de la solution #3)

Ce que dit ce schéma

« En haut à gauche, le poste de développement : Elon et Lucie poussent leur code et ouvrent une PR. GitHub prend le relais : la CI contrôle la qualité, je merge, le CD construit les images Docker, les scanne avec Trivy, prépare le VPS et déclenche Prefect — automatiquement jusqu'à la gate GO. »

« Le VPS est le cœur avec état de la solution : 16 conteneurs Docker. L'entrée du trafic — Caddy et nginx — chiffre, route et limite le débit. L'interface expose l'API et les deux Cockpits. Prefect orchestre 18 flows. La plateforme ML regroupe MLflow, PostgreSQL et MinIO. Et le monitoring centralise métriques et logs des deux environnements. »

« Kubernetes, à droite, est jetable : il sert l'API et le Cockpit public en haute disponibilité, avec 2 répliques et un auto-scaling de 2 à 4 pods pour l'API. Il ne dépend pas du VPS pour servir : au démarrage, il copie depuis S3 le modèle en production et les données dont il a besoin. »

« En bas, S3 : c'est l'archive et le relais. Les données versionnées par DVC, la copie du modèle pour Kubernetes, le certificat HTTPS partagé, les secrets, et les prédictions de Kubernetes, que le VPS importe pour mesurer le drift. Enfin, les cadenas : tout l'administratif passe par le VPN Tailscale — aucun port d'administration n'est exposé sur Internet. »

3.2.1 La stack technique

Couche	Outil	Rôle
Données	DVC + S3 Scaleway	Versionnage des données (remote s3://cac-mlops-data/dvc)
Validation	Pandera	Contrôle des 4 fichiers ONISR sur 3 niveaux
ML	scikit-learn, XGBoost, LightGBM	Benchmark de 3 algorithmes, champion par F1
Tracking	MLflow	Runs, paramètres, métriques, registre de modèles (@Production)
API	FastAPI + JWT	POST /predict · GET /health · GET /metrics
Passerelle	Caddy + nginx	HTTPS, routage, limitation anti-abus à 2 niveaux
Orchestration	Prefect	18 flows, gate manuelle (pause du flow)
CI/CD	GitHub Actions + GHCR	3 workflows (ci, deploy, cleanup), scan Trivy
Monitoring	Prometheus + Loki + Grafana	7 dashboards, 12 alertes email
Drift	Evidently	Dérive des données, de la cible et du trafic réel
Applications	Gradio	Cockpit admin (6 onglets) et Cockpit public (accueil + 3 onglets)
Infrastructure	Scaleway VPS + Kapsule	VPS 4 vCPU / 12 Go ; Kubernetes à la demande (2 nœuds)
Sécurité	Tailscale	VPN maillé : outils d'administration invisibles depuis Internet

Tableau 11 — Stack technique (source : doc « Présentation du projet »)

3.2.2 Où sont le code et les données

Élément	VPS	S3 (Scaleway)	Kubernetes
Code	Copie du dépôt + images	—	Images uniquement
Données ONISR	Copie de travail (ETL, entraînement)	Toutes les versions (DVC)	—
Modèles	Fichiers dans MinIO, registre dans PostgreSQL	Copie du modèle @Production	Chargé au démarrage des pods
Prédictions reçues	Table predictions (PostgreSQL)	Lots déposés par l'API Kubernetes	Déposées sur S3 par lots
Certificat HTTPS	Disque du VPS	Certificat partagé de Kubernetes	Lu et renouvelé par Caddy
Monitoring	Prometheus, Loki, Grafana	—	Prometheus éphémère, logs envoyés au VPS

Tableau 12 — Localisation du code et des données

Ce découpage a une conséquence importante : **si le VPS tombe une fois Kubernetes démarré, l'API et le Cockpit public Kubernetes continuent de répondre**. On perd temporairement le pilotage (Prefect, MLflow, Cockpit admin) et la supervision, mais pas le service.

3.3 Intégration, déploiement et tests

Trois filets de sécurité successifs protègent la production, du code proposé jusqu'à l'utilisateur final :

Moment	Quoi	Volume	Si échec
① Avant le merge	CI : tests unitaires, lint, pip-audit, isolation du blueprint	82 tests (20 API + 62 données)	La PR est bloquée
② Après le merge	CD : détection des changements, build et scan Trivy des images, préparation du VPS, lancement du flow	11 étapes	Le déploiement s'arrête, rien n'est touché
③ Après le GO	Tests fonctionnels : 5 tests API + 3 tests Cockpit public, en interne puis en externe	8 tests × 2 passages, sur VPS puis Kubernetes	Rollback automatique + alerte

Tableau 13 — Les trois niveaux de tests (source : doc « Catalogue des tests »)

Les tests fonctionnels se comportent comme un utilisateur. Côté API : santé, obtention d'un jeton, refus sans jeton (401), prédiction, simulation what-if. Côté Cockpit public, un robot (*radio_client*) clique dans chaque onglet : prédiction, simulation « carrefours → giratoires », carte des points noirs. Le passage **interne** vérifie que l'application fonctionne ; le passage **externe**, par l'adresse HTTPS publique, vérifie que l'utilisateur peut y accéder (DNS, certificat, proxy). Le premier échec désigne donc directement la cause.

Les appels de test portent l'en-tête *X-Synthetic* : ils ne sont jamais comptés comme du trafic réel, ni dans les statistiques d'usage, ni dans le calcul du drift.

3.4 Résilience

	T1 Nouvelles données	T2 Nouveau code	T3 Nouveau blueprint
Validations humaines	1	2	2
Stops (arrêts automatiques)	4	3	3
Rollbacks automatiques	3	5	3
Alertes email	8	7	5
Interruption VPS	~30 s	~1 à 30 s	~30 s
Interruption Kubernetes	0	0	0

Tableau 14 — Mécanismes de résilience par déclencheur (source : doc « Mécanismes de résilience »)

Trois principes guident ces mécanismes :

- **La gate protège toute interruption** : avant le GO, les images sont téléchargées mais aucun conteneur n'est recréé ; un STOP laisse la production strictement inchangée, et en Trigger 3 le blueprint est retiré de *main*.
- **Le VPS d'abord, Kubernetes ensuite** : Kubernetes n'est mis à jour que si le VPS est sain après ses tests ; un échec sur le VPS laisse Kubernetes intact.
- **Zéro interruption sur Kubernetes** : les pods sont remplacés un par un (*maxUnavailable: 0*), un nouveau pod n'est mis en service qu'une fois prêt, et un arrêt en douceur de 5 s évite toute requête perdue. En cas d'échec, Kubernetes revient à l'empreinte exacte de l'image précédente.

3.5 Traçabilité : Git × DVC × MLflow

Pour n'importe quel modèle de la plateforme, on remonte à l'octet près jusqu'aux données qui l'ont produit, et on peut les restaurer à l'identique :

Maillon	Outil	Pour le modèle en production
Données brutes	DVC (S3)	16 fichiers ONISR 2021-2024, 130,6 Mo, jamais modifiés
Dataset préparé	DVC + Git	cumul_2021_2022_2023_2024, commité automatiquement par l'ETL (commit 75edf98)
Entraînement	MLflow	Run 1f5f3724 : paramètres, jeux de données, métriques, modèle
Modèle	Registre MLflow	lgbm_accidents v4, alias @Production

Tableau 15 — Chaîne de traçabilité du modèle en production (source : doc « Lignée des données »)

Le 26/09/2026, les 32 fichiers de données du VPS ont été hachés et comparés aux manifestes DVC stockés sur S3 : **32 sur 32 identiques**. Dans MLflow, un seul alias *@Production* existe toutes familles confondues ; promouvoir un modèle consiste à déplacer cet alias, et chaque promotion est horodatée pour reconstituer l'historique (Prod -1, Prod -2...) — ce qui permet un rollback d'urgence vers la version précédente depuis le Cockpit.

3.6 Monitoring et alertes

3.6.1 Architecture de supervision

Grafana est l'unique point de visualisation et d'alerte. Il lit deux Prometheus (VPS et Kubernetes) et un Loki, qui reçoit les logs des deux environnements et les **événements structurés** écrits par les flows, le Cockpit et le CD (*event=gate_open*, *event=rollback...*). Deux principes de mesure :

- **La disponibilité est mesurée de l'extérieur**, comme un utilisateur : une sonde appelle toutes les 15 s l'adresse HTTPS publique de chacun des 4 accès publics, en traversant tout le chemin réel.
- **Le trafic réel est séparé du trafic de test** : chaque compteur distingue *real* et *test*.

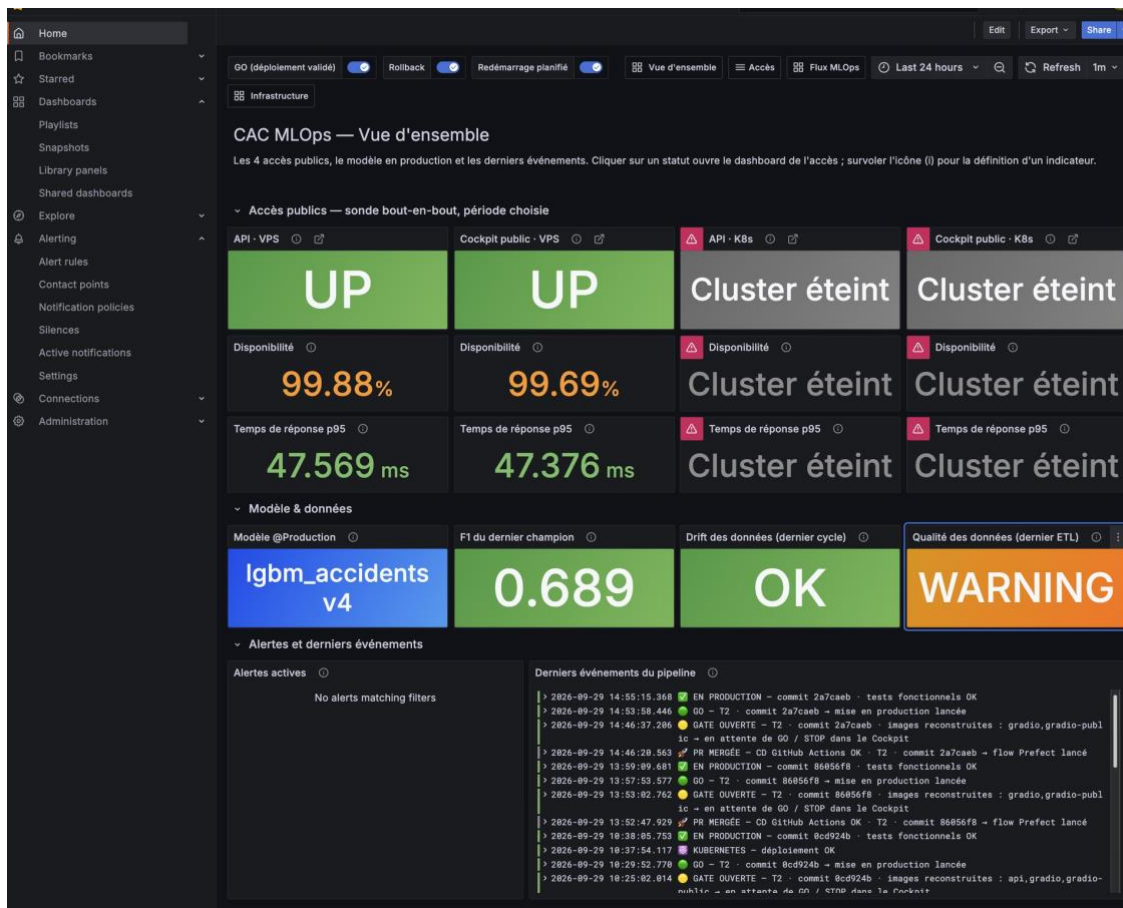


Figure 4 – Grafana : vue d'ensemble (santé des 4 accès publics, modèle, données, événements)

3.6.2 Les 7 dashboards et les 12 alertes

Les 7 dashboards (vue d'ensemble, 4 dashboards d'accès API et Cockpit public sur VPS et Kubernetes, flux MLOps, infrastructure) sont générés par un script, qui produit aussi le catalogue de leurs indicateurs dans la documentation : chaque panneau porte une icône (i) qui explique ce qu'il mesure, comment, et comment le lire.

Source	Alerte	Condition
Prometheus	Brute force (401)	> 20 requêtes rejetées sans jeton en 5 min
Loki	DDoS / rate-limit (429)	> 50 refus 429 par nginx en 5 min
Prometheus	RAM critique / Disque critique	< 10 % de RAM ou < 15 % d'espace /data pendant 2 min
Loki	Prefect — erreur de flow	Ligne ERROR ou CRITICAL dans les logs du worker
Loki	MLOps critique / warning	Échec de déploiement, drift, absence de champion...
Loki	CI/CD échoué, gate refusée, rollback	Événements deploy_pipeline failed, STOP, rollback
Prometheus K8s	Répliques API / endpoint Kubernetes	< 2 pods API ou sonde publique en échec pendant 2 min

Tableau 16 — Les 12 règles d'alerte, regroupées (source : doc « Monitoring »)

3.6.3 La dérive des données (Evidently)

La dérive est surveillée par trois questions, du plus structurel au plus immédiat :

- Les données ont-elles changé ?** — part des 24 variables d'entrée de l'année analysée dont la distribution s'écarte des années d'entraînement (distance de Jensen-Shannon pour les variables catégorielles, de Wasserstein normalisée pour les numériques, seuil 0,1). OK sous 10 %, WARNING au-delà, CRITICAL au-delà de 25 %. Résultat 2024 : **0 variable sur 24**.
- La réalité a-t-elle changé ?** — dérive de la cible : la part d'accidents graves passe de 34,6 % à 34,1 % (stable).

3. **Les demandes reçues ressemblent-elles à l'entraînement ?** — chaque lundi, sur les prédictions réelles des 90 derniers jours (VPS et Kubernetes), avec un minimum de 100 prédictions.

Un enseignement notable du projet : sur un même trafic réel, **100 prédictions donnaient un verdict CRITICAL** (42 % des variables en dérive), alors que **1 000 prédictions donnaient OK** (2 variables sur 24). Sur une variable à forte cardinalité comme le département (≈ 100 modalités), un petit échantillon mesure surtout un « trou » d'échantillonnage. Un indicateur statistique n'a de sens qu'avec un volume suffisant — d'où le refus de conclure sous un minimum, que l'on pourrait relever à 1 000.

Aucun réentraînement n'est déclenché automatiquement par la dérive : les vraies gravités de l'année en cours ne sont connues qu'à la publication suivante. La dérive est un signal pour Léon, avec une alerte email s'il devient critique.

3.7 Sécurité

Menace	Protection
Accès aux outils d'administration	VPN Tailscale : Prefect, MLflow, Grafana, Prometheus, MinIO et le Cockpit admin ne sont joignables que par le tailnet
Interception	HTTPS de bout en bout côté utilisateur (Caddy, certificats Let's Encrypt / ZeroSSL renouvelés automatiquement)
Abus de l'API	Jetons JWT obligatoire ; limitation nginx à 2 niveaux : par client (20/min) et globale (60/min) ; alertes 401 et 429
Vulnérabilités	pip-audit en CI ; scan Trivy des images, bloquant sur une CVE critique ; dépendances épinglées
Changement non maîtrisé	Protection de la branche main (PR + CI obligatoires) ; gate humaine avant toute mise en production
Fuite de secrets	Secrets rotatifs stockés sur S3 et lus à l'usage ; clé S3 dédiée et restreinte pour Caddy ; aucune base exposée sur Internet

Tableau 17 — Mesures de sécurité

3.8 Les applications

La solution livre trois points d'accès : l'**API** pour les applications externes, le **Cockpit public** pour Julie (sur le VPS et sur Kubernetes) et le **Cockpit d'administration** pour Léon (VPS, par VPN).

3.8.1 Le Cockpit d'administration

Six onglets : Accueil, Predict, What-if, Points Noirs, Cockpit et Docs. L'onglet Cockpit regroupe six accordéons : validation des déploiements en attente (GO / STOP avec comparaison des métriques), orchestration des flows, healthcheck des services, dérive des données et qualité, versions des modèles (rollback d'urgence vers la version précédente, rapports de performance), et liens.

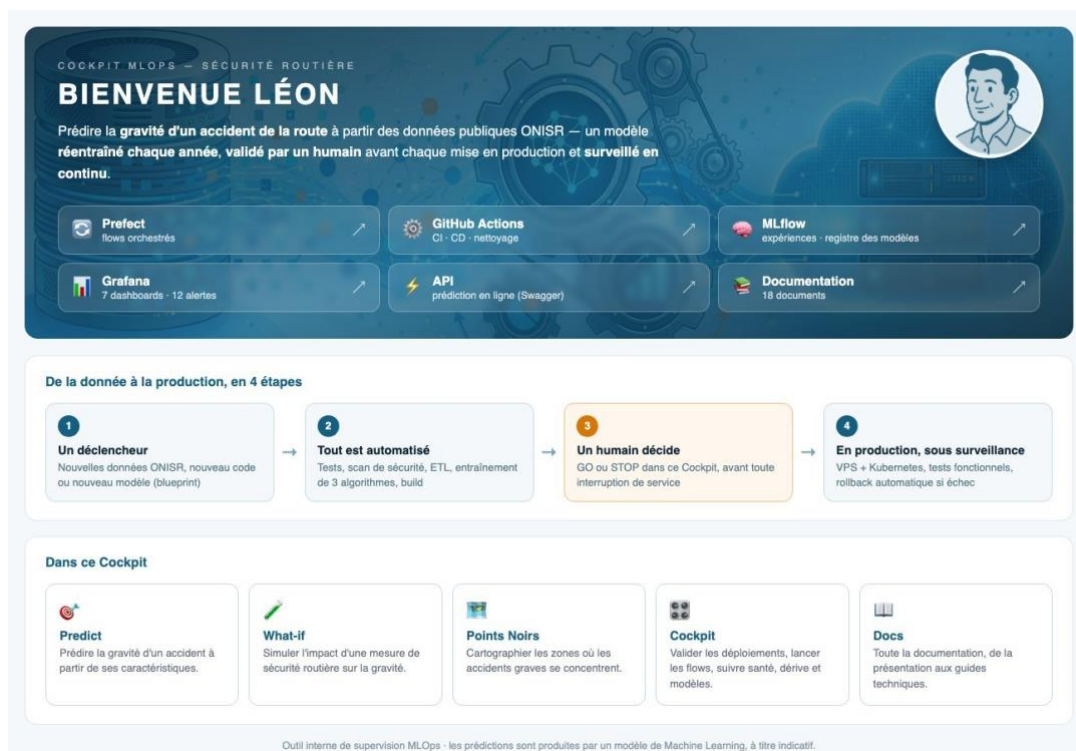


Figure 5 – Cockpit d'administration : accueil « Bienvenue Léon », une tuile par outil

3.8.2 Le Cockpit public

Un accueil « Bienvenue Julie » et trois outils : **Predict** (estimer la gravité d'un accident), **What-if** (simuler une mesure de sécurité sur des milliers d'accidents réels, avec l'explication de la méthode) et **Points Noirs** (carte des zones où les accidents graves se concentrent). Une date « Mise à jour du ... » en bas de l'accueil change à chaque mise en production.



Figure 6 – Cockpit public : accueil « Bienvenue Julie »

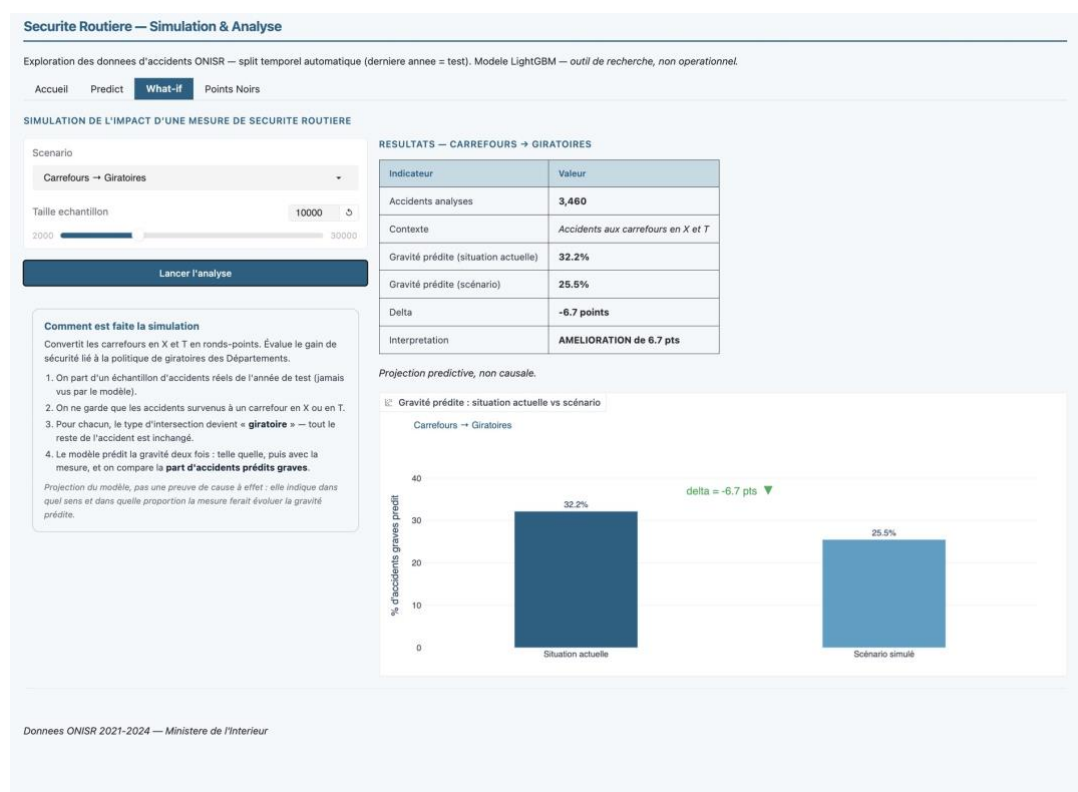


Figure 7— Cockpit public : simulation « Carrefours → Giratoires » (gravité prédite 32,2 % → 25,5 %)

3.8.3 La documentation

L'onglet Docs donne accès à 18 documents, maintenus avec le code (liste en annexe) : ce rapport en est largement issu.

3.9 Démonstrations

Trois démonstrations courtes, scriptées dans *scripts/demo/*, illustrent la solution en direct :

Démo	Ce qu'elle prouve	Durée
Un changement mis en production	Le chemin complet d'un changement (PR → CI → merge → gate → GO → tests) et sa traçabilité ; une date change sur l'accueil des Cockpits	~5 min
Déployer sur Kubernetes sans couper	Les pods sont remplacés un par un ; un témoin interrogeant l'adresse publique 2 fois par seconde ne voit que des réponses 200	~1 min
Une panne de pod sans interruption	Un pod supprimé : l'autre sert toutes les requêtes et Kubernetes en recrée un (auto-réparation)	~30 s

Tableau 18 — Démonstrations (source : doc « Démonstrations live »)

4. Pour aller plus loin

Plusieurs améliorations ont été identifiées au cours du projet, sans être mises en œuvre, pour ne pas fragiliser la production avant la soutenance.

Rendre le VPS remplaçable. Les fichiers des modèles sont stockés dans MinIO, sur le VPS, et le registre MLflow comme les prédictions sont dans son PostgreSQL : ce sont les seuls éléments qui ne sont pas répliqués hors du VPS. Stocker les artefacts MLflow directement sur S3 et sauvegarder chaque jour la base PostgreSQL sur S3 rendrait le VPS entièrement reconstituable. La migration demande de réécrire les chemins des artefacts dans la base MLflow : une opération à mener hors période critique.

Basculer automatiquement entre les deux accès. Le VPS et Kubernetes ont chacun leur adresse ; si le VPS tombe, les utilisateurs doivent changer d'adresse. Un DNS de type failover ou un load balancer devant les deux environnements rendrait la bascule transparente.

Superviser depuis l'extérieur. La supervision est hébergée sur le VPS supervisé : un arrêt du VPS apparaît comme un trou de mesure, sans alerte. Une sonde externe indépendante (service de monitoring tiers) lèverait cette limite.

Mesurer la performance réelle du modèle. Le drift observe les entrées, pas la qualité des prédictions, faute de vraies gravités en temps réel. Rapprocher les prédictions des données ONISR publiées l'année suivante donnerait une mesure a posteriori de la performance en production.

Affiner les indicateurs de dérive. Relever le minimum de prédictions de 100 à 1 000, et calculer un drift par environnement grâce à la colonne *source* (VPS / Kubernetes) déjà enregistrée.

Industrialiser Kubernetes. Un cluster permanent piloté en GitOps (par exemple Argo CD), et des déploiements progressifs (canary, shadow) pour comparer un nouveau modèle sur du trafic réel avant de le promouvoir.

Conclusion

Ce projet montre qu'un modèle de Machine Learning simple — une classification binaire sur 27 variables — demande, pour vivre en production, une chaîne bien plus vaste que le modèle lui-même : ingestion tolérante d'une source publique, validation, versionnage, entraînement comparatif, déploiement contrôlé, tests vus de l'utilisateur, rollback, supervision, sécurité et documentation.

La solution tient ses trois engagements. **Fiabilité** : trois déclencheurs, un seul chemin vers la production, des tests avant et après, une restauration automatique en cas d'échec. **Automatisation** : une nouvelle année ONISR est détectée, chargée, versionnée et transformée en modèle candidat sans intervention ; seule la décision de mise en production reste humaine, par choix. **Sécurité** : administration par VPN uniquement, chiffrement, limitation des abus et scan des images. L'objectif produit — une application toujours disponible pour Julie — est mesuré en continu, de l'extérieur, et tenu par deux environnements de production dont l'un ne s'interrompt jamais pendant un déploiement.

Le principal enseignement est méthodologique : chaque incident réel (fichier source mal nommé, image reconstruite avec une version incompatible, certificat non renouvelé, statistique trompeuse sur un petit échantillon) s'est traduit par un correctif automatisé et une règle documentée. C'est cette boucle — observer, corriger, automatiser, documenter — qui fait la robustesse d'une chaîne MLOps, davantage que le choix de tel ou tel outil.

La suite naturelle serait de rendre le VPS lui-même remplaçable, d'automatiser la bascule entre environnements et de mesurer a posteriori la performance réelle du modèle, pour fermer la boucle entre la prédiction et la réalité observée.

Bibliographie

1. **Sculley, D., et al. (2015).** Hidden Technical Debt in Machine Learning Systems. *Advances in Neural Information Processing Systems*, 28.
2. **Breck, E., Cai, S., Nielsen, E., Salib, M., & Sculley, D. (2017).** The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction. *IEEE International Conference on Big Data*.
3. **Kreuzberger, D., Kühl, N., & Hirschl, S. (2023).** Machine Learning Operations (MLOps): Overview, Definition, and Architecture. *IEEE Access*, 11.
4. **Google Cloud (2020).** MLOps: Continuous delivery and automation pipelines in machine learning. *Cloud Architecture Center*.
5. **Huyen, C. (2022).** *Designing Machine Learning Systems*. O'Reilly Media.
6. **Treveil, M., et al. (2020).** *Introducing MLOps*. O'Reilly Media.
7. **Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016).** *Site Reliability Engineering*. O'Reilly Media.
8. **Breiman, L. (2001).** Random Forests. *Machine Learning*, 45(1), 5-32.
9. **Chen, T., & Guestrin, C. (2016).** XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD Conference*.
10. **Ke, G., et al. (2017).** LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in Neural Information Processing Systems*, 30.
11. **Lin, J. (1991).** Divergence measures based on the Shannon entropy. *IEEE Transactions on Information Theory*, 37(1), 145-151.
12. **ONISR.** Bases de données annuelles des accidents corporels de la circulation routière. data.gouv.fr.
13. **Documentations officielles :** MLflow, DVC, Prefect, Evidently, FastAPI, Gradio, Kubernetes, Prometheus, Grafana Loki, Tailscale.

Annexes

A. La documentation du Cockpit

Document	Contenu
Présentation du projet	Vue d'ensemble, stack, 3 déclencheurs, cycle annuel — point d'entrée
Flux MLOps	Version simplifiée des mécanismes de résilience, pour un public non technique
Architecture globale	Stack VPS, Docker, Prefect, CI/CD, monitoring, Kubernetes
Guide administrateur	Infrastructure, Docker Compose, Tailscale, monitoring, Kubernetes
Mécanismes de résilience	Gates, rollbacks et interruptions par déclencheur
Monitoring	Prometheus, Loki, Grafana : collecte, 7 dashboards, 12 alertes
Dérive des données — Evidently	Les 3 questions de dérive, tests, seuils, lecture des rapports
Entraînement des modèles	Champion, conditions de promotion, prod / dev, blueprint, MLflow
Guide Data Scientist · Guide hyperparamètres	Workflow d'exploration, blueprint, paramètres par algorithme
Guide MLOps Engineer · Guide MLOps Lead	Exploitation, maintenance, gouvernance et gate
Catalogue des tests	CI, CD et tests fonctionnels
Catalogue ETL	Les 4 étapes de la chaîne de données et ses auto-corrrections
Lignée des données	DVC × Git × MLflow, du fichier ONISR au modèle en production
Dictionnaire des données	Les 27 variables et la cible
Runbook — échec pipeline CI/CD	Que faire quand le déploiement échoue avant le flow
Demos live	Les 3 démonstrations de soutenance

Tableau 19 — Documentation disponible dans l'onglet Docs (<https://mlops.jakat-inc.fr/ci-docs/>)

B. Chiffres clés

Indicateur	Valeur
Données	4 années ONISR (2021-2024), 221 048 accidents, 27 variables
Modèle en production	lgbm_accidents v4 — F1 0,689 · AUC 0,837 · rappel 0,767 · accuracy 0,764
Tests	82 tests unitaires en CI · 8 tests fonctionnels × interne / externe × VPS / Kubernetes
Orchestration	18 flows Prefect · 3 déclencheurs · 1 gate humaine
Infrastructure	VPS 16 conteneurs · Kubernetes 2 nœuds, API 2 → 4 pods
Monitoring	7 dashboards · 12 alertes · 4 accès publics sondés toutes les 15 s
Documentation	18 documents

Tableau 20 — Chiffres clés de la solution

C. Glossaire

Terme	Définition
Blueprint	Fichier des hyperparamètres de référence (config/model_params.yml) utilisé par l'entraînement de production
Champion	Meilleur modèle d'un cycle, candidat à la mise en production
Drift (dérive)	Écart entre la distribution des données observées et celle des données d'entraînement
FTe	Tests fonctionnels joués après déploiement, en interne puis par l'adresse publique (externe)
Gate	Point d'arrêt du flow de déploiement, levé par un humain (GO) ou annulé (STOP)
HPA	Horizontal Pod Autoscaler : ajuste le nombre de pods Kubernetes à la charge
@Production	Alias MLflow du modèle servi en production
Rollback	Retour automatique à la version précédente (modèle, images ou blueprint) après un échec
Split temporel	Apprentissage sur les années passées, évaluation sur la plus récente
Trigger	Événement déclenchant une mise en production : nouvelles données, nouveau code, nouveau modèle

Tableau 21 — Glossaire